

Отчет по выполнению 2 и 3 лабораторных работ.

Филонов Иван 8 группа

1. Развертывание базового приложения в K8s

1.1. Введение

Целью данной работы является изучение основ оркестрации контейнеров путём развертывания динамического REST-приложения в кластере Kubernetes. В ходе выполнения решаются задачи по контейнеризации сервиса, публикации образа на Docker Hub и настройке инфраструктуры через декларативные манифесты Deployment, Service и ConfigMap. Итогом работы является организация внешнего доступа к приложению с помощью Ingress-контроллера.

1.2. Ход работы

Для начала описываем базовое приложение на Python, используя Flask. Оно генерирует простую страничку с приветствием.

```
1  import os
2  from flask import Flask
3
4  app = Flask(__name__)
5
6
7  @app.route("/")  @00Fokus00 *
8  def hello():
9      name = os.getenv("NAME", "Guest")
10
11     return f"""
12     <html>
13         <head><title>K8s Hello App</title></head>
14         <body>
15             <h1>Hello {name}</h1>
16             <p>Это базовое приложение для проверки работы Kubernetes.</p>
17         </body>
18     </html>
19     """
20
21
22  if __name__ == "__main__":
23
24      app.run(host="0.0.0.0", port=5001)
```

К этому приложению необходимо написать Dockerfile.

```

1  >> FROM python:3.11-slim
2  WORKDIR /app
3
4  COPY requirements.txt .
5  RUN pip install --no-cache-dir -r requirements.txt
6
7  COPY app.py .
8
9  EXPOSE 5001
10
11 CMD ["python", "app.py"]

```

С помощью команды в терминале мы собираем и публикуем образ на Docker Hub.

```

ps\devops-practice1-docker-compose> cd .\PythonKube\
ps\devops-practice1-docker-compose\PythonKube> docker build -t fokus00/hello-python:v1 .

```

```

ops-practice1-docker-compose> cd .\PythonKube\
ops-practice1-docker-compose\PythonKube> docker push -t fokus00/hello-python:v1 .

```

Далее мы описываем ConfigMap. Здесь мы описываем общие настройки, которые не являются секретными.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-config
data:
  NAME: "Ivan"

```

После этого создаем файл Deployment. Он описывает то, как запускать наши контейнеры.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: hello-python
          image: fokus00/hello-python:v1
          ports:
            - containerPort: 5001
          env:
            - name: NAME
              valueFrom:
                configMapKeyRef:
                  name: hello-config
                  key: NAME

```

Дальше описываем Service. Тут описываются адреса подов.

```

apiVersion: v1
kind: Service
metadata:
  name: hello-service
spec:
  selector:
    app: hello
  ports:
    - protocol: TCP
      port: 80 # Порт, на котором сервис будет доступен внутри кластера
      targetPort: 5001 # Порт, который слушает Python

```

Осталось описать Ingress. Он принимает внешний трафик и отправляет в нужный Service.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-ingress
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: hello-service
            port:
              number: 80

```

На этом описание конфигурационных файлов для лабораторной работы №2 окончено. Осталось все запустить.

Устанавливаем minikube, заходим в командную строку и запускаем командой: minikube start.

```

C:\Users\iafil>minikube start
* minikube v1.38.1 на Microsoft Windows 11 Home Single Language 25H2
* Используется драйвер docker на основе существующего профиля
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Перезагружается существующий docker container для "minikube" ...
! Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Подготавливается Kubernetes v1.35.1 на Docker 29.2.1 ...
* Компоненты Kubernetes проверяются ...
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  - Используется образ gcr.io/k8s-minikube/storage-provisioner:v5
  - Используется образ registry.k8s.io/ingress-nginx/controller:v1.14.3
  - Используется образ registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
  - Используется образ registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
* Verifying ingress addon...
* Включенные дополнения: storage-provisioner, ingress, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.30.5, which may have incompatibilities with Kubernetes 1.35.1.
  - Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
* Готово! kubectl настроен для использования кластера "minikube" и "default" пространства имён по умолчанию

C:\Users\iafil>

```

Дальше нужно применить наши конфигурационные файлы. Делаем следующим образом:

```

PS C:\Users\iafil\OneDrive\Документы\course3\devops\devops-practice1-docker-compose\PythonKube> kubectl apply -f .
configmap/hello-config unchanged
deployment.apps/hello-deployment unchanged

```

Для полноценной работы нам нужно добавить аддон Ingress.

```
C:\Users\iafil>minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  - Используется образ registry.k8s.io/ingress-nginx/controller:v1.14.3
  - Используется образ registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
* Verifying ingress addon...
* The 'ingress' addon is enabled

C:\Users\iafil>
```

Далее в отдельном терминале нужно запустить туннелирование.

```
Администратор: Командная строка - minikube tunnel
Microsoft Windows [Version 10.0.26200.8246]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Windows\System32>minikube tunnel
* Tunnel successfully started

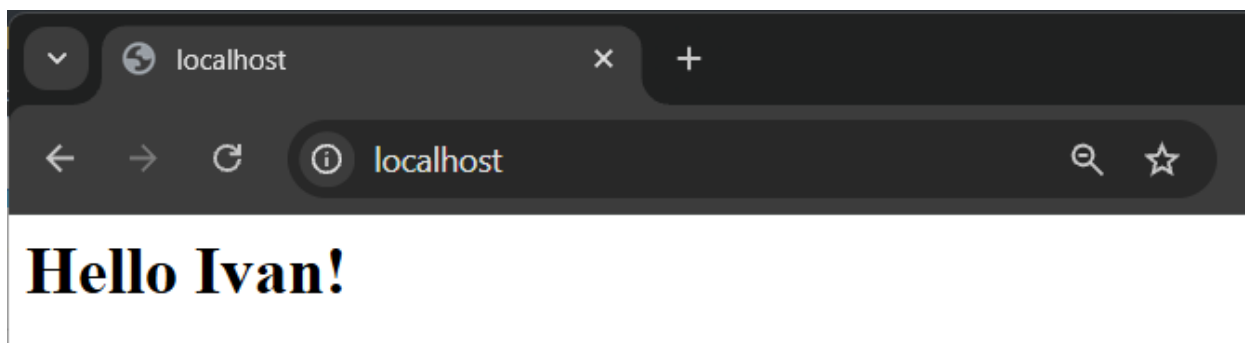
* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...

* Starting tunnel for service envoy-default-my-gateway-1c7c06f0.
! Access to ports below 1024 may fail on Windows with OpenSSH clients older than v8.1. For more information, see: https://minikube.sigs.k8s.io/docs/handbook/accessing/#access-to-ports-1024-on-windows-requires-root-permission
* Starting tunnel for service hello-ingress.
```

Проверяем наш порт

```
C:\Users\iafil>kubectl get ingress
NAME          CLASS    HOSTS      ADDRESS      PORTS      AGE
hello-ingress <none>  *          192.168.49.2 80          3d5h
```

Переходим на localhost:80 и видим что все получилось.



Теперь переходим к выполнению второй части задания.

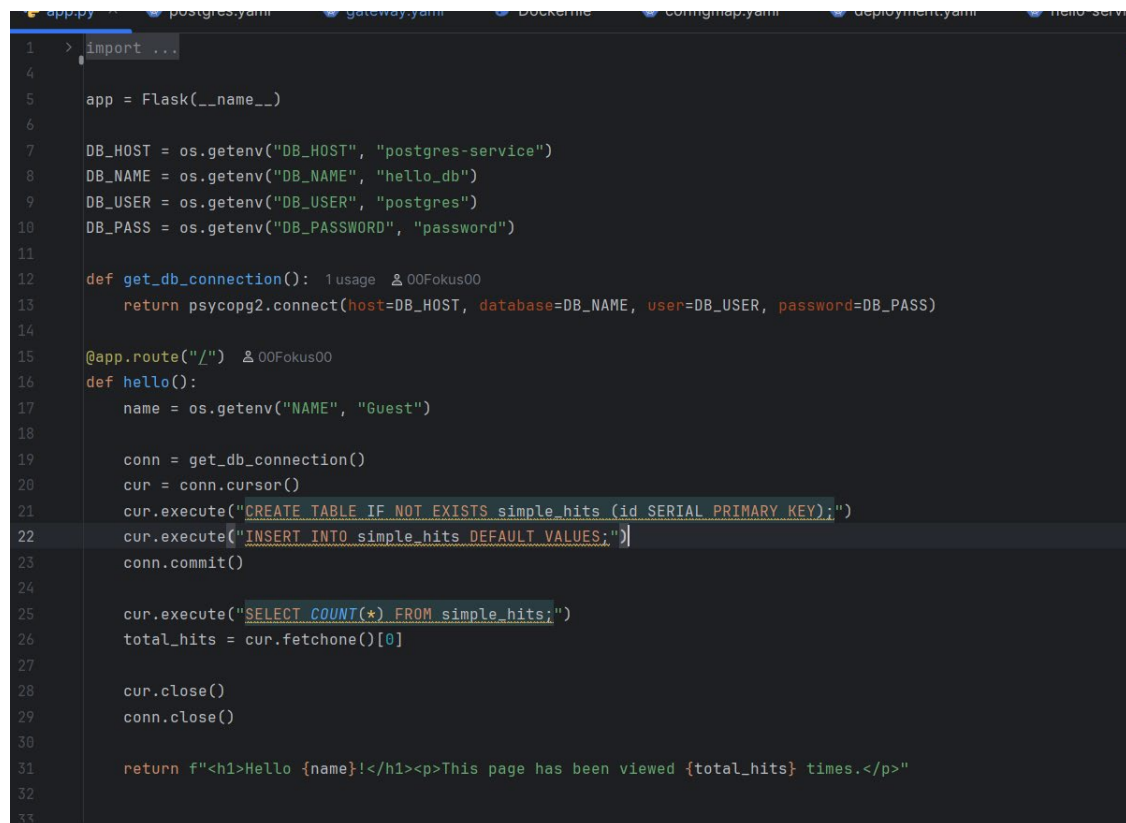
2. StatefulSet, Volumes, GatewayAPI

2.1. Введение

Целью работы является изучение работы с данными и современными методами маршрутизации в Kubernetes. В ходе выполнения задачи реализуется хранение информации в базе данных PostgreSQL с помощью StatefulSet и Volumes, что гарантирует сохранность данных при перезагрузке системы. Также осуществляется переход на Gateway API - новый стандарт управления трафиком, заменяющий Ingress.

2.2. Ход работы

Перепишем наше приложение и добавим туда взаимодействие с базой данных.



```
1 > import ...
4
5 app = Flask(__name__)
6
7 DB_HOST = os.getenv("DB_HOST", "postgres-service")
8 DB_NAME = os.getenv("DB_NAME", "hello_db")
9 DB_USER = os.getenv("DB_USER", "postgres")
10 DB_PASS = os.getenv("DB_PASSWORD", "password")
11
12 def get_db_connection(): 1 usage 2 00Fokus00
13     return psycopg2.connect(host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASS)
14
15 @app.route("/") 2 00Fokus00
16 def hello():
17     name = os.getenv("NAME", "Guest")
18
19     conn = get_db_connection()
20     cur = conn.cursor()
21     cur.execute("CREATE TABLE IF NOT EXISTS simple_hits (id SERIAL PRIMARY KEY);")
22     cur.execute("INSERT INTO simple_hits DEFAULT VALUES;")
23     conn.commit()
24
25     cur.execute("SELECT COUNT(*) FROM simple_hits;")
26     total_hits = cur.fetchone()[0]
27
28     cur.close()
29     conn.close()
30
31     return f"<h1>Hello {name}!</h1><p>This page has been viewed {total_hits} times.</p>"
32
33
```

Описываем postgres.yaml для stateful реализации.

```

apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
stringData:
  POSTGRES_PASSWORD: secretpassword
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432

```

```

9  apiVersion: apps/v1
10 kind: StatefulSet
11 metadata:
12   name: postgres
13 spec:
14   serviceName: postgres-service
15   replicas: 1
16   selector:
17     matchLabels:
18       app: postgres
19   template:
20     metadata:
21       labels:
22         app: postgres
23   spec:
24     containers:
25       - name: postgres
26         image: postgres:15
27         env:
28           - name: POSTGRES_PASSWORD
29             valueFrom:
30               secretKeyRef:
31                 name: postgres-secret
32                 key: POSTGRES_PASSWORD
33           - name: POSTGRES_DB
34             value: hello_db
35         volumeMounts:
36           - name: pgdata
37             mountPath: /var/lib/postgresql/data # Куда монтируем диск
38   volumeClaimTemplates:
39     - metadata:
40         name: pgdata
41       spec:
42         accessModes: ["ReadWriteOnce"]
43         resources:
44           requests:
45             storage: 1Gi

```

Далее необходимо исправить deployment, указать будущую версию образа и указать данные бд.

```
env:
  - name: NAME
    valueFrom:
      configMapKeyRef:
        name: hello-config
        key: NAME
  - name: DB_HOST
    value: "postgres-service"
  - name: DB_NAME
    value: "hello_db"
  - name: DB_USER
    value: "postgres"
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: postgres-secret
        key: POSTGRES_PASSWORD
```

Осталось только описать gateway.yaml. Я прямо описал GatewayClass, без этого не работал Envoy. Gateway описывает точку входа. HTTPRoute описывает куда направить трафик внутри кластера.

```
1  apiVersion: gateway.networking.k8s.io/v1
2  kind: GatewayClass
3  metadata:
4    name: eg
5  spec:
6    controllerName: gateway.envoyproxy.io/gatewayclass-controller
7  ---
8  apiVersion: gateway.networking.k8s.io/v1
9  kind: Gateway
10 metadata:
11   name: my-gateway
12 spec:
13   gatewayClassName: eg
14   listeners:
15     - name: http
16       protocol: HTTP
17       port: 8080
18       allowedRoutes:
19         namespaces:
20           from: Same
21   ---
22 apiVersion: gateway.networking.k8s.io/v1
23 kind: HTTPRoute
24 metadata:
25   name: hello-route
26 spec:
27   parentRefs:
28     - name: my-gateway
29   rules:
30     - matches:
31         - path:
32             type: PathPrefix
33             value: /
34       backendRefs:
35         - name: hello-service
36         port: 80
```

Применяем все изменения.

```
er-compose\PythonKube> docker build --no-cache -t fokus00/hello-python:v2 .
```

```
-compose\PythonKube> docker push --no-cache -t fokus00/hello-python:v2 .
```

```
se\PythonKube> kubectl apply -f .
```

Проверяем получилось ли настроить gateway.

```
C:\Users\iafil>kubectl get gateway
NAME          CLASS    ADDRESS    PROGRAMMED    AGE
my-gateway    eg       127.0.0.1   True          2d11h

C:\Users\iafil>
```

Пробуем запустить через консоль.

```
C:\Users\iafil>minikube service hello-service
```

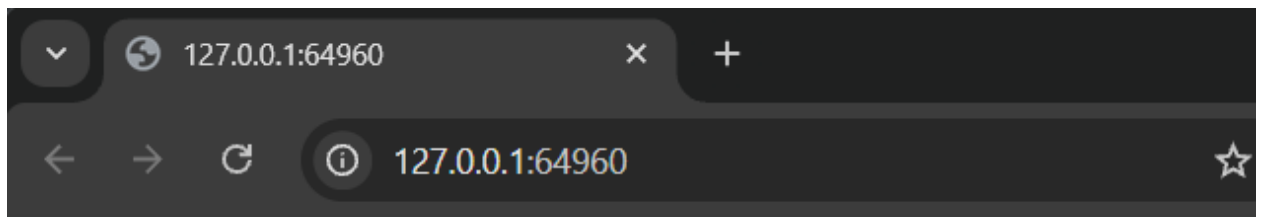
NAMESPACE	NAME	TARGET PORT	URL
default	hello-service		No node port

```
* service default/hello-service has no node port
! Services [default/hello-service] have type "ClusterIP" not meant to be exposed, however for local development minikube
allows you to access this !
* Starting tunnel for service hello-service.
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-service		http://127.0.0.1:64960

```
* Opening service default/hello-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

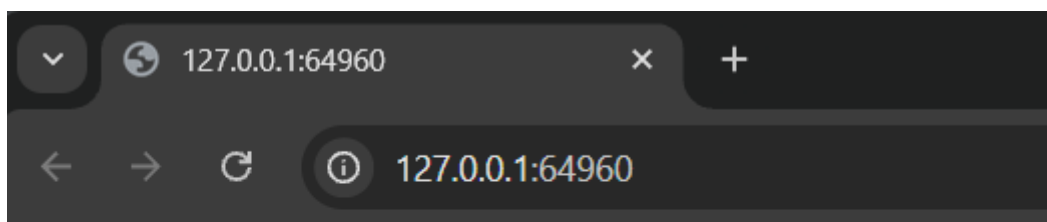
Открывается страница в браузере. Которая хранит кол-во посещений данной страницы.



Hello Ivan!

This page has been viewed 46 times.

При обновлении страницы счетчик обновляется. При перезапуске k8s значение сохраняется.



Hello Ivan!

This page has been viewed 48 times.

3. Вывод

Выполнение работ позволило освоить полный цикл развертывания приложений в Kubernetes. Практическое использование StatefulSet и Volumes решило задачу надежной работы базы данных PostgreSQL, а внедрение стандарта Gateway API обеспечило более гибкое и управление сетевым трафиком. В результате был получен опыт построения масштабируемой и отказоустойчивой архитектуры, где программный код, конфигурации и хранилище данных четко разделены и взаимодействуют между собой.